Méthodes statiques et méthodes de classe

6 exercices — Python

Rappel

	@staticmethod	@classmethod
Premier paramètre	aucun	`cls` : la classe sur laquelle on appelle la méthode
Accès à l'instance	non	non
Accès à la classe	seulement en la nommant en dur	oui, via `cls` (suit l'héritage)
Usage typique	fonction utilitaire rangée dans la classe	constructeur alternatif, état partagé par toutes les instances

Exercices 1 à 3 : **@staticmethod**. Exercices 4 à 6 : **@classmethod** (l'exercice 6 mélange les deux).

Pour chaque exercice, les classes vont dans un package `models/` et le scénario de test dans `main.py`. La sortie doit correspondre exactement au résultat attendu.

Exercice 1 — Méthodes statiques : validateur de formulaire

Énoncé

Écrivez une classe `Validateur` qui regroupe des règles de vérification. Elle n'a ni constructeur ni attribut : on ne l'instancie jamais, toutes ses méthodes sont des `@staticmethod` et renvoient un booléen.

Méthode	Renvoie True si…
`est_email_valide(email)`	exactement un **@**, aucun espace, une partie non vide avant le @, et un domaine qui contient un point, ni au début ni à la fin.
`est_code_postal_valide(code)`	exactement 5 chiffres.
`est_telephone_valide(numero)`	une fois les espaces et les points retirés : 10 chiffres commençant par 0.
`est_mot_de_passe_fort(mdp)`	au moins 8 caractères, dont une majuscule, une minuscule, un chiffre et un caractère spécial (ni lettre ni chiffre).

`valider_formulaire(formulaire)` → `list[str]` : reçoit un dictionnaire `{champ: valeur}` avec les clés `email`, `code_postal`, `telephone` et `mot_de_passe`, et renvoie la liste des champs invalides, dans cet ordre. Un champ absent est invalide. Elle doit **réutiliser** les quatre méthodes précédentes.

Contrainte : pas de module *re* (expressions régulières).

Dans main.py

- Testez chaque règle sur les valeurs du résultat attendu, en appelant les méthodes directement sur la classe (`Validateur.est_email_valide(...)`).
- Validez le **formulaire A** : `alice@exemple.fr`, `69003`, `07 98 76 54 32`, `Soleil#2026`.
- Validez le **formulaire B** : `bob@exemple`, `69003`, `07 98 76`, sans mot de passe.

Résultat attendu

```
E-mails :
  alice@exemple.fr   ✓
  bob@exemple        ✗
  @exemple.fr        ✗
  c d@exemple.fr     ✗
  eve@@exemple.fr    ✗
Codes postaux :
  75001              ✓
  7500               ✗
  75A01              ✗
Téléphones :
  06 12 34 56 78     ✓
  06.12.34.56.78     ✓
  12 34 56 78 90     ✗
  06 12 34           ✗
Mots de passe :
  Azerty12!          ✓
  azerty12!          ✗
  Azerty!!           ✗
  Az1!               ✗
Formulaire A : valide
Formulaire B : champs invalides → email, telephone, mot_de_passe
```

Exercice 2 — Méthodes statiques : convertisseur d'unités

Énoncé

Écrivez une classe `Convertisseur` composée uniquement de méthodes statiques.

Méthode	Formule
`celsius_vers_fahrenheit(c)`	c × 9 / 5 + 32
`fahrenheit_vers_celsius(f)`	(f − 32) × 5 / 9
`celsius_vers_kelvin(c)` / `kelvin_vers_celsius(k)`	c + 273,15 / k − 273,15
`km_vers_miles(km)` / `miles_vers_km(mi)`	km × 0,621371 / mi ÷ 0,621371
`kg_vers_livres(kg)` / `livres_vers_kg(lb)`	kg × 2,20462 / lb ÷ 2,20462

`convertir(valeur, de, vers)` → `float` : conversion générique. Unités reconnues : `"C"`, `"F"`, `"K"`, `"km"`, `"mi"`, `"kg"`, `"lb"`.

- Elle s'appuie sur les méthodes ci-dessus (aucune formule recopiée).
- Entre deux températures, on passe toujours par les degrés Celsius (F → K = F → C puis C → K).
- Même unité au départ et à l'arrivée : la valeur est renvoyée telle quelle.
- Unité inconnue : `ValueError("Unité inconnue : m")`.
- Unités de familles différentes : `ValueError("Conversion impossible : km → kg")`.

Dans main.py

- Appelez directement deux méthodes simples : 100 °C en °F, 42,195 km en miles.
- Lancez les conversions génériques du résultat attendu en capturant les erreurs.
- Faites un aller-retour 37,5 C → K → C et vérifiez qu'on retombe sur la valeur de départ.

Les résultats de conversion sont affichés avec 2 décimales.

Résultat attendu

```
100 °C = 212.0 °F
42.195 km = 26.22 mi
Conversions :
  98.6 F → 37.00 C
  300 K → 80.33 F
  26.2 mi → 42.16 km
  75 kg → 165.35 lb
  20 C → 20.00 C
  5 km → kg : Conversion impossible : km → kg
  12 m → km : Unité inconnue : m
Aller-retour : 37.5 C → 310.65 K → 37.50 C
```

Exercice 3 — Méthodes statiques au service d'une instance : facture

Énoncé

Une classe `Facture` mélange des méthodes d'instance et des méthodes statiques. L'objectif est de repérer ce qui n'a **pas besoin** de `self`.

Constructeur : `Facture(numero, client)`, avec une liste `lignes` vide. Chaque ligne est un tuple `(designation, quantite, prix_unitaire, taux_tva)`.

ajouter_ligne(designation, quantite, prix_unitaire, taux_tva=0.20) : lève une `ValueError` si la quantité est ≤ 0 ou si le prix est négatif.

Méthode statique	Rôle
`arrondir_centimes(montant)`	arrondi commercial à 2 décimales : `2.675` → `2.68`. Attention, `round(2.675, 2)` donne 2.67 (indice : module `decimal`, `ROUND_HALF_UP`).
`calculer_tva(montant_ht, taux)`	montant de TVA, arrondi au centime.
`formater_montant(montant)`	chaîne au format français : `1234567.891` → `"1 234 567,89 €"`.

Méthode d'instance	Rôle
`montant_ligne(quantite, prix_unitaire)`	quantité × prix, arrondi au centime.
`total_ht()`	somme des montants des lignes.
`total_tva()`	somme des TVA **calculées ligne par ligne** (chacune arrondie).
`total_ttc()`	total HT + total TVA.

Dans main.py

- Appelez les trois méthodes statiques seules, sans créer de facture.
- Créez la facture `F-001` pour `Client A` avec : Écran 2 × 249,90 (TVA 20 %), Livre 3 × 32,50 (TVA 5,5 %), Câble 5 × 8,99 (TVA 20 %).
- Tentez d'ajouter « Clé USB » avec une quantité de 0 et capturez l'erreur.
- Affichez les lignes puis les trois totaux.

Résultat attendu

```
round(2.675, 2)  = 2.67
arrondir_centimes = 2.68
formater_montant  = 1 234 567,89 €
Ligne refusée : Quantité invalide pour « Clé USB » : 0
  Écran    2 ×  249,90 € =  499,80 €  (TVA 20 %)
  Livre    3 ×   32,50 € =   97,50 €  (TVA 5.5 %)
  Câble    5 ×    8,99 € =   44,95 €  (TVA 20 %)
  Total HT     642,25 €
  TVA          114,31 €
  Total TTC    756,56 €
```

Exercice 4 — Méthodes de classe : constructeurs alternatifs

Énoncé

Une classe `Produit` se construit normalement avec `Produit(reference, nom, prix, stock)`. On veut pouvoir la créer de plusieurs autres façons, grâce à des `@classmethod` qui renvoient `cls(...)`.

Méthode de classe	Comportement
`depuis_dict(donnees)`	les clés `reference`, `nom`, `prix`, `stock` sont obligatoires ; le prix est converti en `float`, le stock en `int`. Clé absente → `ValueError("Champ manquant : prix")`.
`depuis_chaine(texte)`	format `"REF: Nom du produit xQUANTITE @PRIX"`, par exemple `"SOU03: Souris sans fil x25 @19.99"`. Le nom peut contenir des espaces. Format incorrect → `ValueError("Format invalide : ...")`.
`echantillon(nom)`	produit gratuit (prix 0) en 1 exemplaire, référence `"ECH-"` + les 3 premières lettres du nom en majuscules.

`__str__` affiche : `[REF] Nom — 19.99 € (25 en stock)`.

Créez aussi une sous-classe `ProduitSolde(Produit)` dont le `__str__` ajoute `" [SOLDÉ]"`. Sans rien redéfinir d'autre, `ProduitSolde.depuis_chaine(...)` doit renvoyer un **ProduitSolde** : c'est pour cela qu'on écrit `cls(...)` et jamais `Produit(...)` dans les méthodes de classe.

Dans main.py

- Créez et affichez 4 produits : `Produit("CLV01", "Clavier", 49.90, 12)`, un depuis le dictionnaire `{"reference": "ECR02", "nom": "Écran", "prix": "189.00", "stock": "4"}`, un depuis la chaîne `"SOU03: Souris sans fil x25 @19.99"` et un échantillon `"Casque"`.
- Capturez les erreurs pour un dictionnaire sans prix et pour la chaîne `"Lampe 3 pièces 15 euros"`.
- Créez `ProduitSolde.depuis_chaine("TAP04: Tapis de souris x5 @9.90")` et affichez-le avec le nom de son type, puis affichez le type de `ProduitSolde.echantillon("Webcam")`.

Résultat attendu

```
[CLV01] Clavier — 49.90 € (12 en stock)
[ECR02] Écran — 189.00 € (4 en stock)
[SOU03] Souris sans fil — 19.99 € (25 en stock)
[ECH-CAS] Casque — 0.00 € (1 en stock)
Erreur : Champ manquant : prix
Erreur : Format invalide : 'Lampe 3 pièces 15 euros'
[TAP04] Tapis de souris — 9.90 € (5 en stock) [SOLDÉ] → ProduitSolde
ProduitSolde
```

Exercice 5 — Méthodes de classe : compteur et registre partagés

Énoncé

Chaque `Employe(nom, service)` reçoit automatiquement un matricule unique (`EMP-001`, `EMP-002`, …). Le compteur et le registre des employés sont des **attributs de classe**, partagés par toutes les instances.

Élément	Rôle
`_compteur` (attribut de classe)	nombre d'employés créés, 0 au départ.
`_registre` (attribut de classe)	dictionnaire `{matricule: employe}`.
`_prochain_matricule()`	méthode de classe : incrémente le compteur et renvoie le matricule formaté sur 3 chiffres. Appelée par le constructeur, qui enregistre aussi l'employé.
`nombre_crees()`	renvoie le compteur.
`trouver(matricule)`	renvoie l'employé, ou `None`.
`par_service(service)`	liste des employés de ce service.
`reinitialiser()`	remet le compteur à 0 et vide le registre.

`__str__` affiche : `EMP-001 Alice Martin (Développement)`.

Dans main.py

- Créez Alice Martin (Développement), Karim Benali (Comptabilité), Sofia Rossi (Développement) et Hugo Petit (Ventes), et affichez-les.
- Affichez le nombre d'employés créés, cherchez `EMP-003` et `EMP-099`, puis listez le service Développement.
- **Piège** : exécutez `alice._compteur = 100`. Montrez que cela crée un attribut sur l'instance seulement : le compteur de la classe ne bouge pas.
- Créez Léa Moreau (Ventes), réinitialisez la classe, puis créez Tom Girard (Support).

Résultat attendu

```
  EMP-001 Alice Martin (Développement)
  EMP-002 Karim Benali (Comptabilité)
  EMP-003 Sofia Rossi (Développement)
  EMP-004 Hugo Petit (Ventes)
Créés : 4
EMP-003 → EMP-003 Sofia Rossi (Développement)
EMP-099 → introuvable
Développement : Alice Martin, Sofia Rossi
alice._compteur = 100 | Employe.nombre_crees() = 4
Nouveau : EMP-005 Léa Moreau (Ventes)
Après réinitialisation : 0 | None
Nouveau : EMP-001 Tom Girard (Support)
```

Exercice 6 — Synthèse : héritage, cls et méthodes statiques

Énoncé

Une classe mère `Vehicule(plaque, annee)` et trois sous-classes `Voiture`, `Moto` et `Camion`, qui ne redéfinissent que l'attribut de classe `nombre_roues` (4, 2 et 6). Tous les véhicules créés sont conservés dans une liste `_flotte`, attribut de la classe `Vehicule`.

Méthode	Type	Comportement
`est_plaque_valide(plaque)`	statique	format `AA-123-AA` : 2 majuscules, 3 chiffres, 2 majuscules. Le constructeur l'utilise et lève `ValueError("Plaque invalide : ...")`.
`depuis_ligne(ligne)`	classe	`"AB-123-CD 2021"` → `cls(plaque, annee)`. `Moto.depuis_ligne(...)` renvoie donc une Moto.
`creer(type_vehicule, plaque, annee)`	classe	cherche parmi `cls.__subclasses__()` celle dont le nom correspond (sans tenir compte de la casse). Type inconnu → `ValueError("Type de véhicule inconnu : ...")`.
`flotte()`	classe	les véhicules de la flotte qui sont des instances de `cls`. Sur Vehicule : tous ; sur Moto : seulement les motos.
`nombre()`	classe	`len(cls.flotte())`.

`__str__` affiche le nom du type sur 7 caractères : `Moto  EF-456-GH (2019) — 2 roues`.

Dans main.py

- Testez la validation de plaques, sur Vehicule puis sur Moto (même résultat).
- Créez `Voiture.depuis_ligne("AB-123-CD 2021")`, `Moto.depuis_ligne("EF-456-GH 2019")`, `Vehicule.creer("Camion", "IJ-789-KL", 2015)` et `Vehicule.creer("voiture", "MN-012-OP", 2023)`.
- Capturez les erreurs pour une « Trottinette » et pour la plaque `XX-00-YY`.
- Affichez le type des trois premiers véhicules, la flotte, puis le nombre par classe.

Résultat attendu

```
  AB-123-CD  ✓
  ab-123-cd  ✗
  AB123CD    ✗
  AB-12-CD   ✗
Moto.est_plaque_valide('AB-123-CD') : ✓
  Erreur : Type de véhicule inconnu : Trottinette
  Erreur : Plaque invalide : XX-00-YY
Types : Voiture Moto Camion
Flotte :
  Voiture AB-123-CD (2021) — 4 roues
  Moto    EF-456-GH (2019) — 2 roues
  Camion  IJ-789-KL (2015) — 6 roues
  Voiture MN-012-OP (2023) — 4 roues
Nombre : Vehicule: 4 | Voiture: 2 | Moto: 1 | Camion: 1
```